COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURES AND ALGORITHM	**SECTION:**	CPE21S1
DATE PERFORMED:	18/8/2026	**DATE SUBMITTED:**	26/8/2026
NAME:	LOREEN JAE D. PILAPIL	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

ILO A:

Screenshot

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> // will be used for our seeding function

const int max_size = 50;

int main() {
    //generate random values
    int dataset[max_size];
    srand(time(0));
    for(int i = 0; i < max_size; i++){···

//show your datasets content
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    return 0;

}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\loreen\Documents\CPE21S1\CPE010\HOA6P1>  ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\
loreen\.vscode\extensions\ms-vscode.cpptools-1.33.8-win32-x64\debugAdapters\bin\WindowsDebug
Launcher.exe' '--stdin=Microsoft-MIEngine-In-0ikalipf.zge' '--stdout=Microsoft-MIEngine-Out-
ndiciywy.5p0' '--stderr=Microsoft-MIEngine-Error-hok5nijg.z15' '--pid=Microsoft-MIEngine-Pid
-dywmlde5.s04' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
22605 363 3190 32721 12601 24522 7210 7550 2582 30403 19659 7225 2123 2101 649 4612 25120 18
514 23457 13236 2665 27309 22822 27297 16233 1427 1281 29346 241 16287 14647 5177 12744 3162
5 27754 20234 26703 30271 10147 2194 12901 2871 2382 4885 20617 4867 31110 8408 11326 4134
PS C:\Users\loreen\Documents\CPE21S1\CPE010\HOA6P1>
```

Observation	The code outputs 50 random values for a data set. It uses the srand(time(0) to make the random values and the for loop stored values for each element in the data set.

Table 6-1. Data Generated and Observations

Code

```cpp
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> // will be used for our seeding function
#include "searching.h"

const int max_size = 50;

int main() {

    int dataset[5] = {10, 20, 30, 40, 50};

    int item;
    std::cout << "number to search: ";
    std::cin >> item;

    if (linearSearch(dataset, max_size, item)) {
        std::cout <<"Searching is successful";
    }
    else {
        std::cout <<"Searching is unsuccessful";
    }


    return 0;

}
```

```
#ifndef SEARCHING_H
#define SEARCHING_H

template <typename T>
bool linearSearch(T data[], int N, T item) {
    int i = 0;

    while (i < N) {
        if (item == data[i]) {
            return true;
        }
        else {
            i++;
        }
    }
    return false;
}

#endif
```

Output	`number to search: 10` `Searching is successful`
Observation	The code successfully performed a linear search for arrays. It searches for the entered number then checks the elements one by one.

Table 6-2a. Linear Search for Arrays

Code

```
    //create linked list for linear search
    Node<char> *name1 = new_node('l');
    Node<char> *name2 = new_node('o');
    Node<char> *name3 = new_node('r');
    Node<char> *name4 = new_node('e');
    Node<char> *name5 = new_node('e');
    Node<char> *name6 = new_node('n');


    //linked list
    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = NULL;

    //show linked list
    Node<char>* current = name1;


    while (current != NULL) {
        std::cout << current->data << " ";
        current = current->next;
    }

    std::cout << std::endl;


    //linear search
    if (linearLS(name1, 'n')) {
        std::cout <<"Searching is successful";
    }
    else {
        std::cout <<"Searching is unsuccessful";
    }

    return 0;
}
```

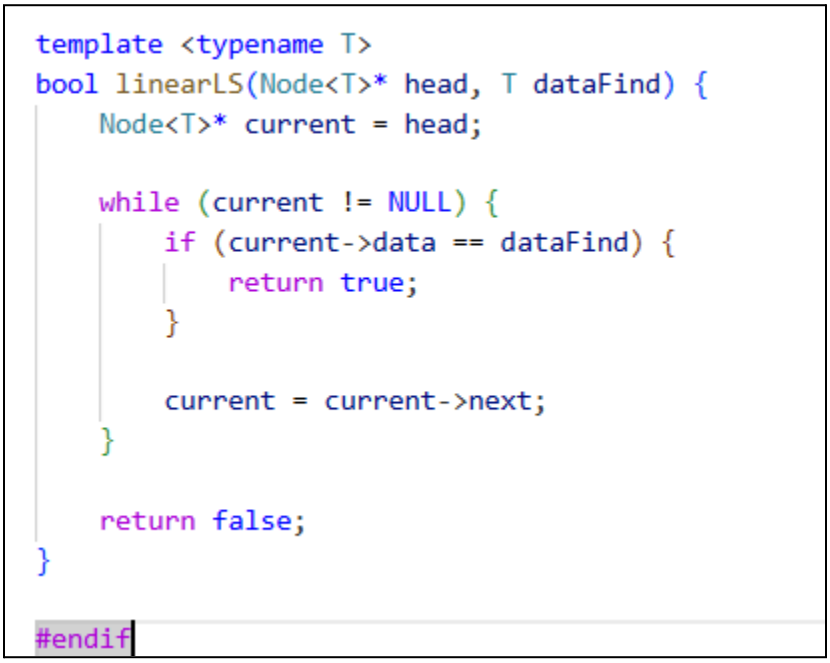
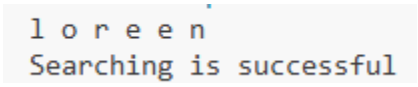
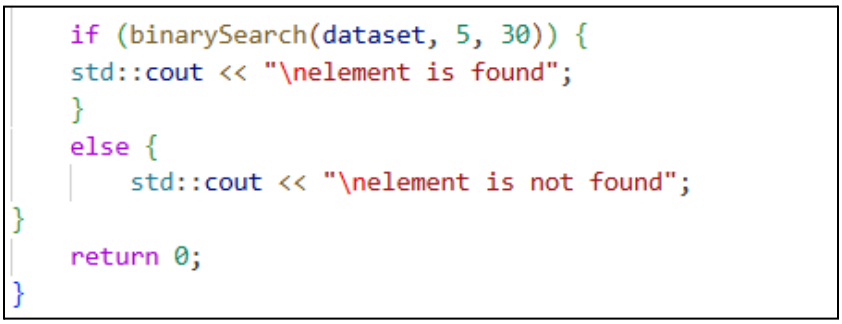
	
Output	
Observation	The code successfully performed a linear search for linked list. It creates a linked list with my name then connects each letter to the next. Then it displays it all. It's successful after it finds the letter 'n'.

Table 6-2b. Linear Search for Linked List

Code	

```
template <typename T>
bool binarySearch(T arr[], int n, T item) {
    int low = 0;
    int up = n - 1;

    while (low <= up) {
        int mid = (low + up) / 2;

        if (item == arr[mid]) {
            return true;
        }

        else if (item < arr[mid]) {
            up = mid - 1;
        }

        else {
            low = mid + 1;
        }
    }
    return false;
}
#endif
```

Output

```
number to search: 10
Searching is successful
l o r e e n
Searching is successful
element is found
```

Observation

The code successfully performed a binary search to find ‘30’ .

Table 6-3a. Binary Search for Arrays

Code

```
Node<int>* node1 = new_node(10);
Node<int>* node2 = new_node(20);
Node<int>* node3 = new_node(30);
Node<int>* node4 = new_node(40);
Node<int>* node5 = new_node(50);

node1->next = node2;
node2->next = node3;
node3->next = node4;
node4->next = node5;
node5->next = NULL;

Node<int>* head = node1;
std::cout << "\nLinked List: ";

Node<int>* current2 = head;

while (current2 != NULL) {
    std::cout << current2->data << " ";
    current2 = current2->next;
}
std::cout << std::endl;
int searchItem;

std::cout << "Enter number to search: ";
std::cin >> searchItem;

if (binarySearchLL(head, searchItem)) {
    std::cout <<"element is found" << std::endl;
}
else {
    std::cout <<"element is not found" << std::endl;
}
```

```cpp
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last) {
    if (start == NULL) {
        return NULL;
    }
    Node<T>* slow = start;
    Node<T>* fast = start;

    while (fast != last && fast->next != last) {
        fast = fast->next;

        if (fast != last) {
            fast = fast->next;
            slow = slow->next;
        }
    }
    return slow;
}
```

```
template <typename T>
bool binarySearchLL(Node<T>* head, T key) {
    Node<T>* start = head;
    Node<T>* last = NULL;


    while (start != last) {
        Node<T>* middle = getMiddle(start, last);

        if (middle == NULL) {
            return false;
        }
        if (middle->data == key) {
            return true;
        }
        else if (middle->data > key) {
            last = middle;
        }
        else {
            start = middle->next;
        }
    }

    return false;
}
```

Output

```
number to search: 10
Searching is successful
l o r e e n
Searching is successful
element is found
Linked List: 10 20 30 40 50
Enter number to search: 20
element is found
```

Observation	The code successfully performed a binary search for a linked list. It searched for the number '20' and successfully found it.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the Questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons Would it be necessary to find the key '18'?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.
51

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

PROBLEM 1:

CODE

```cpp
#include <iostream>
#include "nodes.h"
#include "searching.h"

int main() {

    int dataset[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    int key = 18;
    int comparisons = 0;
    std::cout <<"ARRAY" << std::endl;

    for (int i = 0; i < 10; i++) {
        comparisons++;

        if (dataset[i] == key) {
            std::cout <<"element found: " << dataset[i] << std::endl;
            break;
        }
    }
    std::cout <<"No. of comparisons: " << comparisons << std::endl;

    Node<int>* node1 = new_node(15);
    Node<int>* node2 = new_node(18);
    Node<int>* node3 = new_node(2);
    Node<int>* node4 = new_node(19);
    Node<int>* node5 = new_node(18);
    Node<int>* node6 = new_node(0);
    Node<int>* node7 = new_node(8);
    Node<int>* node8 = new_node(14);
    Node<int>* node9 = new_node(19);
    Node<int>* node10 = new_node(14);

    node1->next = node2;
    node2->next = node3;
    node3->next = node4;
    node4->next = node5;
    node5->next = node6;
    node6->next = node7;
    node7->next = node8;
    node8->next = node9;
    node9->next = node10;
    node10->next = NULL;

    Node<int>* current = node1;
    comparisons = 0;
```

	``
	std::cout <<"\nLINKED LIST" << std::endl;
	
	while (current != NULL) {
	    comparisons++;
	
	    if (current->data == key) {
	        std::cout <<"element found: " << current->data << std::endl;
	        break;
	    }
	
	    current = current->next;
	}
	std::cout <<"No. of comparisons: " << comparisons << std::endl;
	
	return 0;
	``

OUTPUT	``` ARRAY element found: 18 No. of comparisons: 2 LINKED LIST element found: 18 No. of comparisons: 2 ```
ANSWER	2 comparisons for both array and linked list approach

PROBLEM 2:

CODE

```cpp
#include <iostream>

int sequentialSC(int arr[], int size, int k) {
    int count = 0;

    for (int i = 0; i < size; i++) {
        if (arr[i] == k) {
            count++;
        }
    }
    return count;
}

int main() {
    int dataset[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int k;

    std::cout <<"search element: ";
    std::cin >> k;

    int count = sequentialSC(dataset, 10, k);

    std::cout <<"No. of repeating instances: " << count << std::endl;

    return 0;
}
```

OUTPUT

```
search element: 18
No. of repeating instances: 2
```

PROBLEM 3:

CODE

```cpp
#include <iostream>

int main() {
    int data[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int key = 8;
    int low = 0;
    int high = 9;
    int step = 1;

    while (low <= high) {
        int mid = (low + high) / 2;

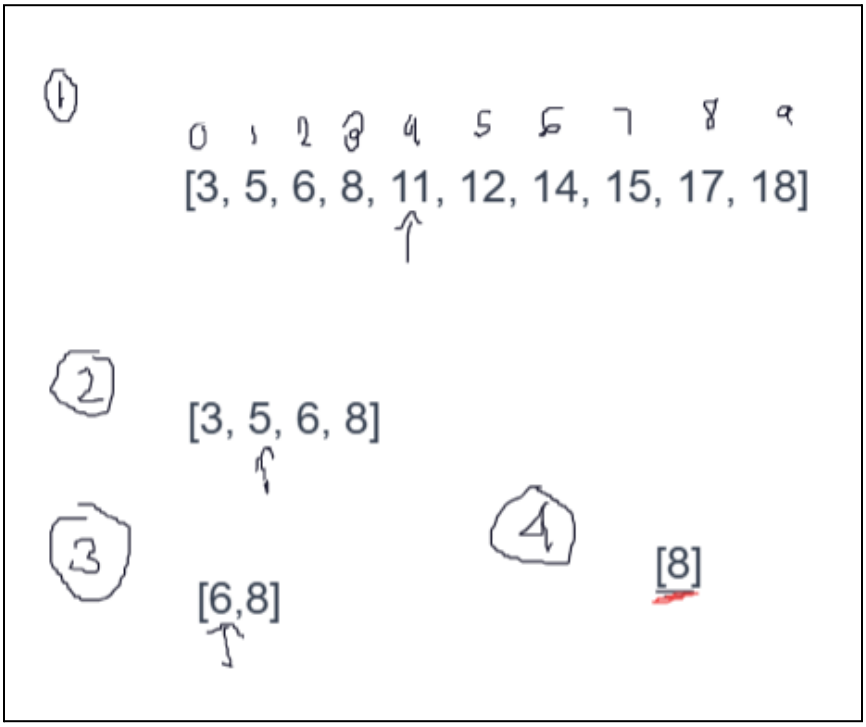
        std::cout <<"step " << step << ": ";
        std::cout <<"middle = " << data[mid]<<std::endl;

        if (data[mid] == key) {
            std::cout <<"8 is found" << std::endl;
            break;
        }
        else if (key < data[mid]) {
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }

        step++;
    }
    return 0;
}
```

OUTPUT

```
step 1: middle = 11
step 2: middle = 5
step 3: middle = 6
step 4: middle = 8
8 is found
```

ANSWER	

PROBLEM 4:

CODE

```
#include <iostream>

bool binarySearch(int data[], int low, int high, int key) {

    if (low > high) {
        return false;
    }
    int mid = (low + high)/2;
    std::cout <<"middle= "<< data[mid]<< std::endl;

    if (data[mid] == key) {
        return true;
    }
    else if (key < data[mid]) {
        return binarySearch(data,low, mid - 1, key);
    }
    else {
        return binarySearch(data, mid + 1,high,key);
    }
}

int main() {
    int data[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int key = 8;

    if (binarySearch(data, 0, 9, key)) {
        std::cout <<"8 is found"<<std::endl;
    } else {
        std::cout <<"8 is not found"<<std::endl;
    }

    return 0;
}
```

OUTPUT

```
middle= 11
middle= 5
middle= 6
middle= 8
8 is found
```

CONCLUSION:

The procedure helped me understand how each technique works. Creating and testing the codes showed me the difference between arrays and linked lists. On the other hand, the Supplementary activity helped me through solving problems. It was a bit difficult but it made me understand the search algorithm better.

The activity taught me linear and binary search in arrays and linked lists. The activity was successful and helped me improve my understanding of searching algorithms in C++. However I believe that I need more practice to confidently do this without assistance, since I found it a bit hard to grasp the idea and how it works.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: - **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: - **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:
- **Helped in debugging and explaining what went wrong**
- **Used to understand the concept better and how it worked**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.